Palyt: Engineering Intern Task

Palyt builds software for restaurants. This task is a small slice of something our product actually does, shrunk to an afternoon. We expect it to take **3 to 4 hours**. If it runs much longer, stop and send us what you have.

A kitchen tracks its ingredients. The menu does not know about any of it. So a dish stays on the menu after the kitchen has run out, someone orders it, and a waiter walks back to the table to apologise.

You are building the kitchen's side of that, and the bit that joins it up.

Two files are attached:

- `stock.json`: what the kitchen holds, and the par level for each ingredient
- `recipes.json`: the menu, with each dish, its price, and what one portion uses

Par is the level a kitchen wants to stay above, not zero. It is the buffer that says "order more of this".

Part 1: manage the stock

The main screen. A list of every ingredient with its quantity and par level, and the usual things you would expect to do to it.

- **See** the list, and find things in it. Fifteen rows today, three hundred next year.
- **Add** a new ingredient.
- **Edit** one. Changing what is in stock is the common case, because deliveries arrive. Changing the par level should also be possible.
- **Delete** one.

The form needs to stop people entering nonsense. You decide what nonsense means here and tell us what you settled on.

Deleting is the interesting one. Two dishes use cashews. Nothing uses bay leaves. Decide what should happen in each case and why. There is more than one defensible answer and we are interested in which you pick.

Part 2: join it to the menu

A second view showing the menu as a diner would see it: the dishes, their prices, and whether each can be ordered right now.

A dish is unavailable when any ingredient it uses has fallen below its par level.

Then make one order work end to end:

1. A diner orders a dish
2. The dish is broken into ingredients using its recipe
3. Those come out of stock
4. Anything now below par takes its dishes off the menu

Put the two views where you can see both at once if you can. Watching one order take a dish off the menu is the point of the whole thing.

Everything in Part 1 should feed this. Restock an ingredient and its dishes come back. Raise a par level and dishes go away without anyone eating anything.

If you run short of time

In this order. Stop wherever you get to and say so.

1. The stock list, and editing a row
2. An order deducting ingredients and the menu reacting
3. Add and delete
4. Search, and validation on the form
5. Anything else you fancy

The git workflow below is not on this list because it is not a step at the end. Do it from the first commit and it costs you nothing.

We would much rather see the first two finished properly than all five half-built.

Scope, so this does not run away

You do not need any of this: QR scanning, payment, accounts, login, a database, tables, seats, order history, or a real backend. Keep the data in memory or write back to the JSON file, whichever you prefer.

Plain HTML and JavaScript is completely fine. If you already know React or Vue and it is faster for you, use it. We are not testing framework knowledge and would rather you did not spend an hour on build setup. Python or JavaScript for the logic, whichever you prefer, and you do not need both.

Tests for the logic. Not the whole app, just the parts that work out what is available and what to deduct.

Three things worth knowing

Stock is measured how you buy it, recipes are written how you cook. You buy paneer by the kilo and a recipe asks for 180 grams. Get that wrong and your numbers are off by a factor of a thousand, so check a couple by hand.

The data is close to a real kitchen's, so some things will not line up neatly. Where you have to make a call, make the one you think is right and tell us. There is no single correct answer to some of them, and noticing is the part we care about.

And if you think a rule we gave you is wrong, say so. The availability rule in Part 2 is written the way the product currently works. That does not make it right. If you find a case it handles badly, we would rather hear about it than have you implement it faithfully and stay quiet. This is not a trick question. It is the most useful thing you could tell us.

How we would like you to work

Doing this as you go costs nothing. Retrofitting it at the end is miserable, which is itself the lesson.

- **Put it on a public GitHub repo.**
- **Commit as you go, in small steps.** One commit per piece of work that stands on its own. "Stock list renders from the JSON file", then "Editing a quantity updates the menu". Not one commit at the end with everything in it. We would honestly rather see eight small commits with one thing broken than a single perfect one, because the first tells us how you work and the second tells us nothing.
- **Write commit messages that mean something.** Say what changed and, when it is not obvious, why. "Fix bug" and "update" tell the next person nothing. The next person is often you, three weeks later.
- **Open a pull request** from a branch into your own `main`, and leave it open rather than merging it. The PR description is a good place for your write-up.
- **Check it in the browser before you call it done.** Tests passing and the thing working are different claims. We have shipped a feature with a fully green test suite that saved nothing to the database, so this one is not theoretical.
- **AI tools are fine.** Say what you used them for, and be ready to explain any line of your code.

What to send

- a link to your GitHub repo, with the pull request open
- a screenshot or short recording of an order taking a dish off the menu

- a README with the command to run your tests, and how to open the page
- a short write-up, half a page is plenty. The PR description is a good home for it

---

The write-up

1. **The calls you made.** Where did the data or the rules not settle it, what did you decide, and why? What should happen when you delete an ingredient a recipe needs?
2. **How you checked.** How do you know your numbers are right? What would have to be wrong for your tests to still pass?
3. **Another day.** What would you fix or build next?

What we are evaluating

- **Does the loop work.** Order a dish, watch the right amounts leave stock, watch the menu react.
- **Does the CRUD work.** Add, edit, delete, and the list still makes sense afterwards.
- **Are the numbers right.** And can you show us they are.
- **What you did with the parts that were not clean.** Spotting them matters more than the specific call you made.
- **Readable code.** Sensible names, small functions, something another person can pick up. Not clever.
- **Your tests.** Would they fail if the logic broke?
- **How you worked.** Your commit history and your PR, as much as the finished code.
- **How you write.** The write-up is part of the task.

Constraints

- **Do not make it pretty.** Clear beats polished. We are not judging your design.
- **Do not research restaurant software.** Common sense is enough.
- **Do not build past the scope above.** Finishing a small thing beats half-building a big one.

Good Luck!